

AI-Assisted URL Shortener

Final Engineering Report — Engineer-Led AI-Assisted Software Engineering Assessment

Prepared by Shrihari Koushik • 2026-06-28

1. Executive Summary

This project is a working prototype that turns a software requirement into a reviewable engineering outcome using **engineer-led, AI-assisted execution** — not autonomous orchestration. It pairs a production-style FastAPI URL-shortener service with an engineering-evidence engine that, for any submitted requirement, derives the understanding, task plan, codebase impact, AI-assisted execution record, quality gates, risks, and a downloadable final summary. Each of the eight assessment requirements (R1–R8) is driven by the actual requirement text, executed where execution is claimed, and covered by automated tests.

Verification status: 33 automated tests pass (`python -m pytest -q`) and static analysis is clean (`python -m ruff check .`). With an OpenAI key configured the LLM paths engage; with no key, deterministic capability-based fallbacks produce the same structured outputs so the system is fully runnable offline.

2. How to Run and Demonstrate

Prerequisites

- **Python 3.11+** is required (the service uses `datetime.UTC`).
- **Revoke and replace the key in** `.env` before running or sharing. To run fully offline, set `AI_SHORTENER_OPENAI_ENABLED=false`.

Start the application and UI

```
cd ai_assisted_shortener
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --host 127.0.0.1 --port 8010
```

Open `http://127.0.0.1:8010/` for the reviewer UI and `http://127.0.0.1:8010/docs` for the interactive API.

Run the automated checks

```
python -m pytest -q # expect: 33 passed
python -m ruff check . # expect: All checks passed!
```

UI walkthrough

- **Shortener:** set Target URL + Custom endpoint (optionally Max clicks / Expiry) → Create link; open the `/r/<code>` link to redirect; Refresh to see analytics.
- **Guardrails:** Max clicks = 1 then open twice → blocked (429); past Expiry → blocked (410); custom endpoint 'api' → rejected (reserved).
- **Engineering:** choose a Scenario, edit the Requirement and Engineer notes → 'Generate without approval' shows a waiting status; 'Engineer approve' / 'Business approve' produce a reviewable outcome; download `summary.md` and the JSON evidence.
- **Implementation:** 'Build real implementation package' → View generated UI / Download package (includes generated code, tests, docs, and `VALIDATION.json`).

3. Architecture Overview

- `app/main.py` — FastAPI routes and HTTP boundary.
- `app/schemas.py` — Pydantic request/response contracts (incl. per-step engineer actions).
- `app/url_service.py` — core shortener rules (create, resolve, expiry, max-click, disable).
- `app/database.py` — SQLite persistence.
- `app/engineering.py` — requirement-driven evidence engine (understanding, decomposition, codebase reasoning, AI phases, risks, assumptions/limitations, summary).
- `app/ai_assist.py` — OpenAI JSON helper (used when a key is configured).
- `app/quality.py` — executable quality gates (ruff subprocess + in-process runtime smoke test).
- `app/implementation.py` — isolated package generation with real validation and a corrective retry.
- `app/codegen.py` — generates code/schema modules, runnable pytest files, and docs; executes the generated tests.
- `tests/` — 33 unit + integration tests.

4. Requirement Coverage (R1–R8)

Req	What it requires	How it is met (and where to verify)
R1	Requirement understanding: intent, ambiguity, normalization	Derived from the input text (AI or heuristic). Two different requirements yield different normalized problems and ambiguity. UI: Evidence panel. API: POST /engineering/execute.
R2	Task decomposition with dependencies and sequencing	Task graph built from detected capabilities with real <code>depends_on</code> ordering; a narrow change yields a focused graph. See <code>task_decomposition</code> in the evidence/summary.
R3	Brownfield codebase reasoning	Per-change impacted files, API, data flow and regression risk (<code>codebase_reasoning.impact_analysis</code>); differs by requirement and no longer mislabels unrelated flows.
R4	AI-assisted execution + quality gates	Six SDLC phases are requirement-specific; gates actually run (ruff + runtime smoke test with duration); security gate blocks sensitive text; sign-off enforced.
R5	Engineering output: code, API/schema, tests, docs	<code>codegen</code> produces dependency-free code modules, runnable pytest, and docs into the package and executes the generated tests (<code>validation.generated_tests</code>).
R6	Validation and risk control	Requirement-specific risks/trade-offs/failure scenarios, each with a mitigation; real guardrails (gates, security, sign-off, package validation).
R7	Controlled oversight: engineer leads and approves	Engineer notes steer capability detection; per-suggestion accept/edit/reject via <code>engineer_actions</code> flows to traceability; code generation produces nothing without sign-off.
R8	Final engineering summary	Downloadable <code>summary.md</code> renders Plan, Trade-offs & Rationale, Artifacts, Quality Gates (Validation), Risks, Assumptions, Limitations, Final Summary.

5. Test Results

All 33 tests pass. They are grouped by area below; service tests are unit-level, the rest are API/integration tests through FastAPI's TestClient with OpenAI disabled (the deterministic path).

Core service & API (6)

- create/redirect/stats; reserved-endpoint rejection; expiry blocked & not counted; max-click limit enforced.

R1–R8 evidence (27)

- R1: understanding derived from input; vague input flags ambiguity; understanding feeds traceability.
- R2: decomposition is requirement-specific; dependencies/sequencing valid; vague input clarifies first.
- R3: codebase reasoning is requirement-specific; reports API and regression risk.
- R4: quality gates actually run; security gate blocks sensitive requirement; six phases requirement-specific.
- R5: generated package is validated; generated output includes code, tests and docs.
- R6: risks/trade-offs are requirement-specific (each risk carries a mitigation).
- R7: engineer notes steer execution; engineer can override AI step actions; sign-off gating (engineering & implementation).
- R8: final summary includes all required sections; assumptions/limitations follow the requirement.
- Scope control: QR, Facebook-clone, Spotify and unknown-product requests are routed to scope review, not silently built.

6. Known Limitations & Recommended Next Steps

Reported honestly so reviewers are not surprised:

- **Secret hygiene:** a live OpenAI key is present in `.env` and must be revoked; it should never be committed.
- **Offline path is rule-based:** without a key, R1–R3/R6/R8 use deterministic capability templates rather than open-ended LLM reasoning; iterative refinement only engages on the AI path.
- **Maintainability:** `app/engineering.py` is large (~1.2k lines) and would benefit from being split into focused modules.
- **Scope/prototype bounds:** single-node SQLite, no authentication model, and no production observability backend.

7. Appendix — Key Endpoints

- `POST /api/links`, `GET /r/{code}`, `GET /api/links/{code}/stats`, `POST /api/links/{code}/disable`
- `POST /engineering/execute`, `GET /engineering/runs/{id}`, `GET /engineering/runs/{id}/summary.md`
- `POST /implementation/execute`, `GET /implementation/runs/{id}/preview`, `GET /implementation/runs/{id}/download`